RxInsight: How It Works

Updated 29 October 2026. Code reviewed: `da15d2c`. This guide supersedes the earlier study PDFs and describes the repaired implementation.

1. The project in one minute

RxInsight is a pharmaceutical commercial analytics warehouse. Python generates structured synthetic extracts, lands them in PostgreSQL staging, validates records, preserves HCP history and loads a star schema. Four SQL reports answer prescriber-ranking, brand-share, territory-attainment and sales-call questions.

The central engineering problem is preserving meaning as data moves. A doctor changing territory must not move old prescriptions into the new territory. A malformed record must become an explained reject rather than disappear in a join.

This is a full-refresh demonstration, not an incremental production feed, clinical system or model-serving application. All commercial data is synthetic.

> Analogy: a company archive has a receiving room, an organised historical record and several reading desks. Staging is the receiving room; the warehouse is the archive; reports are different views of the same record.

2. Verified results

Check	Measured result
Prescription inputs	2,230,247
Loaded / rejected prescriptions	2,229,747 / 500
Loaded sales calls	68,619
Distinct HCPs / historical versions	2,000 / 2,038
Closed HCP versions	38
Default dependency suite	67 passed; zero skips
Compatibility dependency suite	67 passed; zero skips
Independent report checks	12 passed
Source/warehouse aggregate mismatches	Zero

The repaired full-size ETL took 81.0 seconds in the audit environment, including 58.7 seconds for fact copying and foreign-key revalidation. GitHub CI also passed both dependency configurations on a smaller dataset. Timings describe this run, not a guarantee for another machine.

3. Follow the code

File	Responsibility
`etl/config.py`	Paths, database URL, seed, volume and batches
`etl/generate_data.py`	HCP, prescription and call extracts
`etl/load_staging.py`	CSV/DataFrame COPY and constraint handling
`etl/transform.py`	Types, calendar, SCD2 and temporal resolution
`etl/pipeline.py`	Stage order, batch loads, rejects and reconciliation
`sql/analytics/`	The four reports
`etl/measure.py`	Indexing benchmark

The orchestrator creates schemas, copies source files to staging, builds small dimensions, folds HCP versions, then loads facts in batches. Reads use SQLAlchemy and a server-side cursor; writes use psycopg2 COPY.

The default fact batch is 250,000 rows. `chunksize` controls pandas batches, while `stream_results=True` prevents the driver from first buffering the complete result client-side. This bounds the fact-read path, not necessarily the generator or every reject-accumulation path.

4. Declare the schema and grain

HCP means healthcare professional, used here as a prescriber. TRx is total prescriptions and NRx is new prescriptions. A dimension describes context; a fact contains measurements at a declared grain.

Table	One row means
`dim_date`	A calendar day
`dim_product`	A product code and its descriptive attributes
`dim_territory`	A sales territory
`dim_hcp`	A version of an HCP's tracked attributes
`fact_prescriptions`	HCP version, product and day
`fact_sales_calls`	A recorded call about a product to an HCP

Both facts share the dimensions. This is one conformed star schema, not six separate schemas. Joining raw prescriptions directly to raw calls can multiply rows because the facts have different grains. Aggregate to compatible units first.

> Analogy: linking every receipt to every store visit by the shopper can count each purchase several times. Decide whether the question concerns purchases, visits or shoppers per month before writing the join.

The source natural key `hcp_id` follows a person across time. The warehouse surrogate `hcp_key` follows one version. Both are useful, but they answer different questions.

5. SCD Type 2, worked through

Suppose HCP-1 belongs to territory A until 31 May and B from 1 June. The old version is closed and a new current version is added. A March prescription resolves to the old key; an August prescription resolves to the new key. Each fact stores its historically correct territory.

Intervals are inclusive. A new version on 1 June closes its predecessor on 31 May. The open interval ends at `9999-12-31`. Python date comparisons preserve that sentinel on pandas 2.2 and 3.1; forcing it into pandas 2.x nanosecond timestamps previously overflowed.

A partial unique index permits at most one current version per HCP. A CHECK rejects inverted intervals. Tests additionally check overlaps and fact dates against matched history. No single one of these checks proves all the others.

Tracked changes concern territory and decile. An identical tracked state is a no-op. Conflicting same-day or backdated tracked changes raise an error; an explicit backfill process would be needed to rewrite history safely. That backfill is not implemented.

> Analogy: a person keeps their identity after moving house, but an old delivery receipt keeps its original address. Person identity and address-version identity should not be substituted for each other.

6. Validation and quarantine

Staging columns are TEXT so raw values reach validation before typed fact loading. Missing, fractional, non-finite and out-of-range integer values are quarantined. PostgreSQL input validation guards date casts during calendar-boundary calculation, so 30 February cannot abort that earlier stage before quarantine.

A known HCP with no valid history version must still become a reject. The repaired resolver finds valid candidates, then joins them back to every original input position. Overlapping matching versions are rejected rather than selected arbitrarily.

Rejects use positional `iloc`, because earlier filtering can leave gaps in index labels. A label/position mistake can swap a clean row with a malformed row while preserving counts. Check identities as well as this equation:

```
staged_rows == loaded_rows + rejected_rows
```

All 250 numeric rejects in the full run really contained `N/A`; all 250 unknown-product rejects really contained `UNKNOWN-99`.

7. Loading and transaction boundaries

COPY streams buffered records efficiently instead of issuing a separate INSERT command for each row. During fact loading, eight foreign keys are removed and restored with validation inside the fact transaction. An invalid reference prevents successful restoration and cannot be committed as an orphan fact.

Fact reconciliation happens before the fact commit. The complete refresh is not one atomic transaction: schema, staging and dimension stages have earlier commits. The normal command resets the staging and warehouse schemas, so use the project's isolated database.

> Analogy: opening a loading gate temporarily is safe only if every pallet is checked before the delivery is accepted. That does not mean all earlier warehouse operations belonged to the same transaction.

8. Interpret the reports

Prescriber ranking

Own-brand TRx is grouped by HCP surrogate key and territory. RANK gives equal volumes equal rank; NTILE divides ordered rows into approximately equal-sized groups within a territory. The output has 2,038 version rows, not 2,000 unique people.

Stored decile increases toward high volume, while descending NTILE starts at 1 for high volume. Do not compare their raw values as if their directions and definitions matched.

Brand share

The denominator is all tracked products in the same therapeutic area and month. With 60 own-brand and 40 competitor cardiology TRx, share is 60%. Adding 100 unrelated endocrinology TRx should not turn cardiology share into 30%.

```
SUM(trx) OVER (PARTITION BY year_month, therapeutic_area)
```

LAG compares a brand with its preceding reported month in that area. Competitors remain in the denominator. The full report has 192 rows.

Territory attainment

Actuals use own-brand TRx. The target is the average of the first six observed monthly totals plus 8%. This is a derived demonstration quota, not an imported commercial plan. The first six months are explicitly labelled baseline months.

Multiply before dividing: `ROUND(SUM(actual_trx) * 1.08 / COUNT(*))`. For 8,675 prescriptions over six months, the exact target is 1,561.50 and rounds to 1,562. Rounding an intermediate repeating decimal previously produced the wrong boundary result. YTD totals reset by territory and calendar year. The report has 1,200 rows.

Call effectiveness

Exposure follows stable `hcp_id`, so a call before a transfer can precede the same person's prescription afterward. The window includes days 1 through 14 before the prescription and excludes its own day. The generator uses the same boundaries.

The unit is one person-month across all SCD versions. If decile changes, the month uses the decile valid on its last prescribing day. Months without prescribing are absent. The report compares all own-brand activity; it does not isolate each product's promotional effect.

Cohort	Person-months	Mean own-brand TRx
Called	39,225	39.32
Not called	7,959	23.70

An independent raw-CSV as-of calculation reproduced the 47,184 person-months. The reported 65.9% association is observational and based on generated data, not a causal estimate or a guaranteed upper bound on one.

9. Read the indexing evidence honestly

Workload	No secondary indexes	Indexed
Selective prescriber lookup	131.640 ms	2.250 ms
Full aggregate	1,165.269 ms	1,172.546 ms

The selective lookup used a Bitmap Heap Scan and improved about 58.5x in this run. The full aggregate stayed a Parallel Seq Scan with no material improvement. The benchmark discards a warm-up and takes the median of three runs per state. Its full aggregate is a simplified scan/group query, not the complete window report.

`idx_rx_hcp` supports selective prescriber access. The date/product covering index supports suitably filtered workloads; it does not promise that the unfiltered brand report avoids the heap. Selectivity and measured plans justify an index, not table size alone.

10. Reproduce and learn

```
make venv
make up
make generate
make load
make test
make report
make tune
```

PostgreSQL uses host port 5544. `make test` requires a populated database; missing DB checks fail rather than quietly skip. Pure units can run separately on `test_transform.py` and `test_regressions.py`. Report commands stop on SQL errors.

Default requirements pin pandas 3.1.0 and SQLAlchemy 2.1.6. The compatibility file tests pandas 2.2.3 and SQLAlchemy 2.0.55. SQLAlchemy explicitly selects the installed psycopg2 driver in both.

Study order: declare grains; explain history; trace one good row and one reject; reproduce a report by hand; compare a selective and full-scan plan; explain transaction limits. Future work includes incremental watermarks, restartable publication, history backfills, a real quota feed and workload-driven partitioning. None of those, or sharding, is currently implemented.

Sources: Reviewed implementation | Verification record | Implementation CI | Interview questions and tool primer